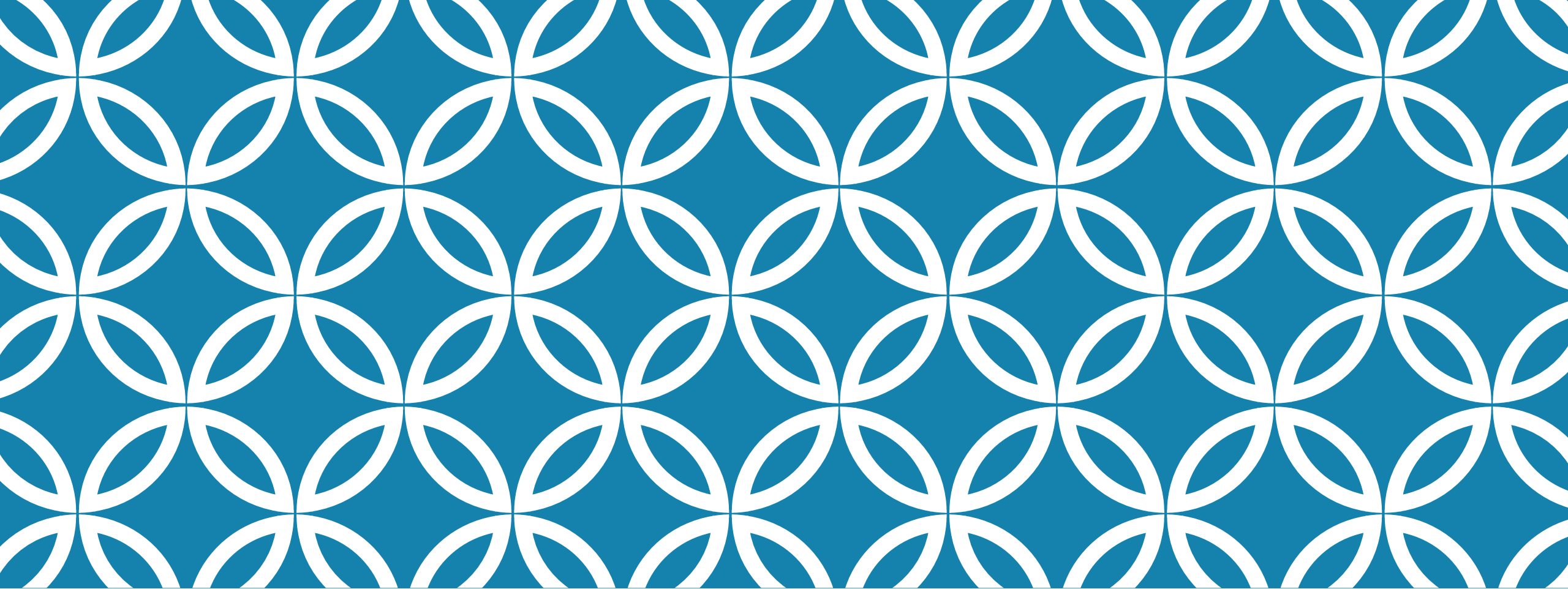




RCPP 02 — RCPP SUGAR

dr Maria Kubara, WNE UW

Rcpp SUGAR

- ❖ „Rcpp sugar” is created with functions in Rcpp namespace in C++ which look like the functions which you know from R
- ❖ Rcpp sugar functions are developed by the authors of Rcpp package
- ❖ They make writing in C++ „sweeter” for R programmer
- ❖ Important: **sugar functions aren't always as fast as C++ original functions**, but they make it much easier to speed up the R code with C++ implementation without being a super fluent C++ programmer

Rcpp SUGAR

❖ all the basic arithmetic and logical operators are vectorised in Rcpp - they will work on vectors like in R)

`+, *, -, /, <, <=, >, >=, ==, !=, !`

❖ one can use summary functions on vectors in the same way as they are used in R

`sum(v), mean(v), median(v), sd(v), sign(v), sqrt(v), pow(v, n)` and many others

Sources with Rcpp sugar functions:

https://teuder.github.io/rcpp4everyone_en/210_rcpp_functions.html#list-of-r-like-function

<https://thecoatlessprofessor.com/programming/cpp/unofficial-rcpp-api-documentation/#sugar>

<https://gallery.rcpp.org/articles/sugar-for-high-level-vector-operations/>

<http://dirk.eddelbuettel.com/code/rcpp/Rcpp-sugar.pdf>

Function to take all the vector
elements to the second power

Rcpp SUGAR EXAMPLE

```
cppFunction("NumericVector squareCppVec(NumericVector x) {  
    int n = x.size();  
    NumericVector result(n);  
    for(int i = 0; i < n; i++) {  
        result[i] = x[i] * x[i];  
    }  
    return result;  
}")
```

Typical C++

This requires a loop

```
cppFunction("NumericVector squareCppVec2(NumericVector x) {  
    return x * x;  
}")
```

Rcpp sugar -> multiplication is vectorised
(works on vectors like in R)

RCPP SUGAR

- ❖ you can easily include error handling in Rcpp code with functions like `warning()` and `stop()`
- ❖ functions like `is_na()` and `na_omit()` allow for easy missing data handling in Rcpp sugar codes
- ❖ additional functions with logical operators: `is_false()`, `is_true()`, `all()`, `any()`, `ifelse(vector, operation1, operation2)` help with navigating the logic of Rcpp code

Sources with Rpp sugar functions:

https://teuder.github.io/rcpp4everyone_en/210_rcpp_functions.html#list-of-r-like-function

<https://thecoatlessprofessor.com/programming/cpp/unofficial-rcpp-api-documentation/#sugar>

<https://gallery.rcpp.org/articles/sugar-for-high-level-vector-operations/>

<http://dirk.eddelbuettel.com/code/rcpp/Rcpp-sugar.pdf>

Rcpp SUGAR EXAMPLE

Calculate MAPE – mean absolute prediction error. Single measure out of a vector of real values and forecasts

```
cppFunction('double mapeCpp(NumericVector real, NumericVector forecasts) {  
  int nf = forecasts.size();  
  int nr = real.size();  
  if (nf != nr) {  
    stop("The length of real and forecasts must be the same.");  
  }  
  double mape = 0.0;  
  for(int i = 0; i < nf; i++) {  
    // use abs for integer and fabs for double values  
    // these two functions work on scalars  
    mape += fabs(real[i] - forecasts[i])/real[i];  
  }  
  return mape/real.size();  
'})
```

Typical C++

This requires a loop to compare all elements and get the measure

```
cppFunction('double mapeCppSugar(NumericVector real,  
                                NumericVector forecasts) {  
  if ( forecasts.size() != real.size() ) {  
    stop("The length of real and forecasts must be the same.");  
  }  
  return mean(abs(real - forecasts)/real);  
  // abs() is a function from Rcpp sugar which works on vectors  
'})
```

Rcpp sugar -> we get the value from vectorised operations on vectors

We also have abs() rcpp sugar function which returns absolute values on the whole vector

C++ VS R – REFERENCING AND COPYING

- ❖ C++ is a language which heavily utilizes „reference” semantics
- ❖ This means that by default values which are to be reused in some other place in the code are not COPIED, but rather kept just in their original place, while the REFERENCE to them is assigned to the code which re-uses them
- ❖ If you want to create an independent copy of the values (like in R), in C++ you need to use clone() function (copying the values, instead of referencing to the original object).
`new_place_storing_the_values = clone(original_values)`

C++ VS R – REFERENCING AND COPYING

- ❖ Example: creating a DataFrame from a set of vectors
- ❖ In R data frame will get copies of the vectors – changes made to the vectors are not changing the values in the data frame – these objects exist independently due to copying.
- ❖ In C++ reference will be used, whenever possible. This means that the data frame will store just the “reference” to its column data (like path to the original vector). This means that the changes made to the original vector... change the values stored in the data frame (as data frame only shows, what these vectors are).

C++ REFERENCING FOR SORTING

- ❖ Referencing is used in many places in C++ - one of the examples is the way `sort()` function is working (sort from `std::` namespace in C++)
- ❖ Unlike in R we are not calling `sort()` (or `arrange()`) on a full vector, but rather we are showing the “references” to where the beginning and the end of the vector is within computer’s memory space. Then C++ is sorting this place in memory,

```
std::sort(myVector.begin(), myVector.end());
```

C++ FOR SIMULATIONS

- ❖ Simulations - running simulations is crucial in machine learning
- ❖ Simulation is re-running the same code with different (random) samples to find the best value (best: minimizing error or loss function or maximizing likelihood in ML).
- ❖ Running the same code numerous times needs very fast loop processing – something that makes C++ shine the most
- ❖ Another example is bootstrapping (repetitive drawing of random samples of the same size as the whole dataset with replacement) – common solution in statistics to find the true value of some estimate for a population using a (limited) sample size
 - ❖ In our example in the script, you can see how very similar code for bootstrapping exercise runs 45 times faster than R version.

COMPLEX OBJECTS IN RCPP SUGAR

❖ In R we usually create more complex output objects from functions than just vectors

❖ thanks to Rcpp sugar we can do the same in Rcpp code – return lists of dataframes

Of course you need to remember to define the type of the object your function in C++ returns

```
// [[Rcpp::export]]  
DataFrame colCVsCpp_df2(NumericMatrix x) {
```

```
// DataFrame object can be created with DataFrame::create() function  
// column names can be assigned with Named() function or _[].  
DataFrame result = DataFrame::create(Named("means") = clone(means_x),  
                                     _["sds"] = clone(sds),  
                                     _["cvs"] = clone(colCVs),  
                                     |  
                                     _["cvs2"] = colCVs);
```

Remember that C++ uses references! If you want to get copies of the vectors you use clone()

```
// List object can be created with List::create() function  
// element names can be assigned with Named() function or _[].  
List result = List::create(Named("means") = means_x,  
                           _["sds"] = sds,  
                           _["cvs"] = colCVs);
```

```
// or just:  
// List result = List::create(means_x, sds, colCVs)  
// if names do not have to be assigned manually
```